

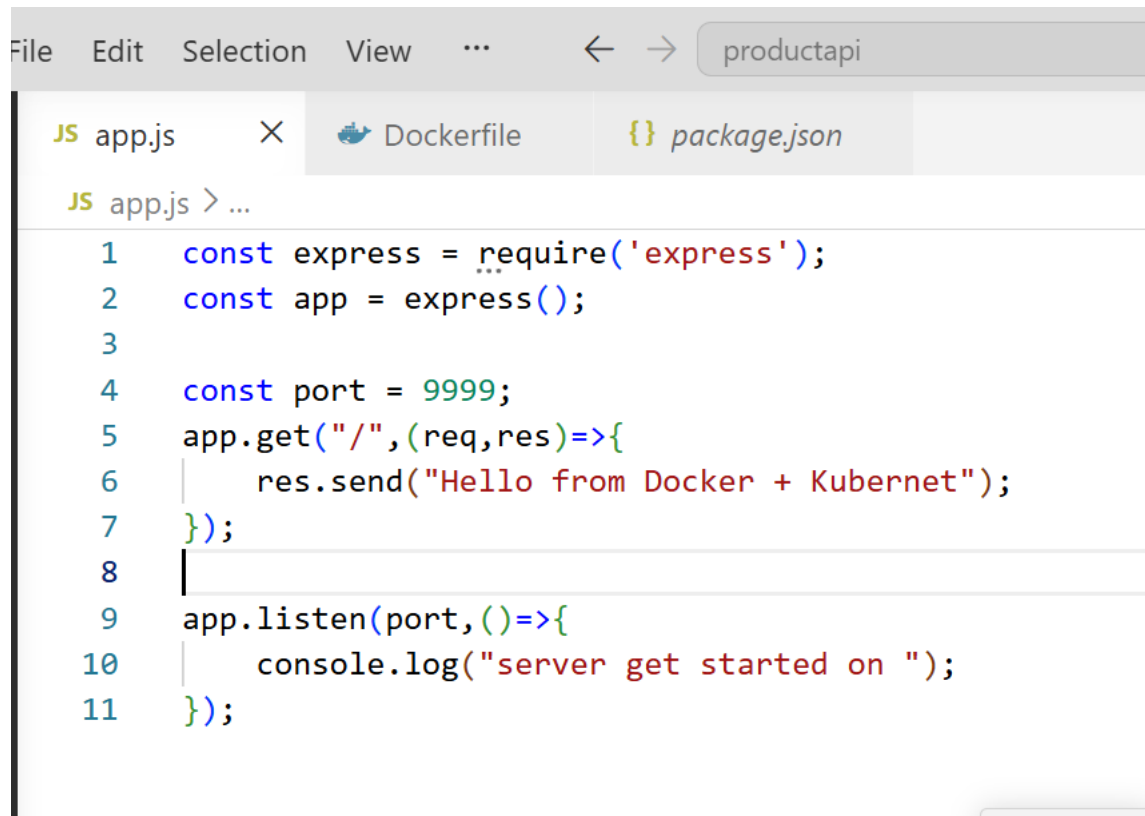
Nodejs project with docker

Step1 : create folder and open in terminal

Step2: in terminal write **npm init -y** it will generate package.json file

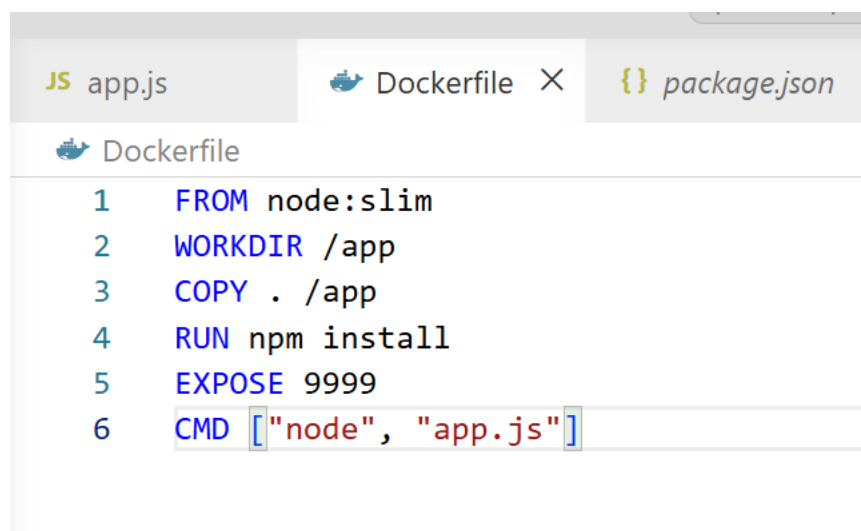
Step3: install express by writing command **npm install express**

Step4: create app.js file and create server by using express

A screenshot of a code editor window titled 'productapi'. The editor has three tabs: 'app.js' (selected), 'Dockerfile', and 'package.json'. The 'app.js' tab shows the following JavaScript code:

```
1  const express = require('express');
2  const app = express();
3
4  const port = 9999;
5  app.get("/", (req, res) => {
6    res.send("Hello from Docker + Kubernetes");
7  });
8
9  app.listen(port, () => {
10    console.log("server get started on ");
11  });
```

Step5: create Dockerfile file with name **Dockerfile only** and write code in it

A screenshot of a code editor window showing the 'Dockerfile' tab. The editor has three tabs: 'app.js', 'Dockerfile' (selected), and 'package.json'. The 'Dockerfile' tab shows the following Dockerfile instructions:

```
1  FROM node:slim
2  WORKDIR /app
3  COPY . /app
4  RUN npm install
5  EXPOSE 9999
6  CMD ["node", "app.js"]
```

Step6: execute app.js file by writing command **node app.js** and go to browser url and write **localhost:9999** to get output and **open docker-destop and docker hub which is necessary for further operation**

Step7: stop server By Pressing **Ctrl+C** and delete **node_modile** folder

Step8: write command to create docker-image from dockerfile

```
docker build -t panchashilw/productapi:0.0.1.RELEASE .
```

Step9: write a command to create docker container from docker-image

```
docker container run -d -p 3000:9999 panchashilw/productapi:0.0.1.RELEASE
```

Step10: go to browser url and write **localhost:3000** to display output

Step11: **docker image ls** // display docker image list

```
docker container ls // display docker list
```

step12: stop container by writing command

```
docker stop container-id
```

step13: Tag image

The syntax for tagging is:

```
docker tag [SOURCE_IMAGE]:[TAG] [NEW_IMAGE_REPOSITORY]:[NEW_TAG]
```

Command :

```
docker tag panchashilw/productapi:0.0.1.RELEASE panchashilw/productapi:0.0.1.RELEASE
```

Above command creates a new tag, **panchashilw/productapi:0.0.1.RELEASE** (this is New) that points to the existing **panchashilw/productapi:0.0.1.RELEASE** (this is Old) image. It does not duplicate the image, but adds a reference so you can push it to your **Docker Hub** account.

Step14: Push docker-image by writing command

```
docker push panchashilw/productapi:0.0.1.RELEASE
```